```
  Serial.print("analog value = " );
  Serial.println(analogValue, DEC);

  if ((analogValue > minReading) && (analogValue < fanStart))
  analogWrite(fanPin, fanStart);
  else analogWrite(fanPin, analogValue);

  delay(250);
}
```

Source Code Summary

Like nearly all of our sketches, we begin this one with a declaration of variables for the pins that we will use, giving them descriptive names. In this case, `fanPin` and `sensorPin`.

```
const int fanPin = 5;
const int sensorPin = A0;
```

Our next block of variables pertain to some general thresholds that relate to our sensor readings and what it takes to get the fan spinning.

```
const int minReading = 25;
const int maxReading = 400;
const int fanStart = 100;
```

The first variable `minReading` can be adjusted to reflect the minimum reading that our sensor is providing. Likewise, `maxReading` is used for the same purpose on the other end of the spectrum. This is, after all, a telematic breath, not telematic gale-force wind. To make the sensor even more sensitive, you might try a smaller value for `maxReading`, like 200. The last variable, `fanStart`, is the minimum value that it takes to get our particular fan up and spinning. Yours may be different.

```
int analogValue = 0;
```

Our last variable is the container we will use to store and manipulate our analog value obtained from reading our analog input pin.

```
void setup() {
  Serial.begin(9600);
}
```

The `setup()` function is a little sparse in this sketch. I think I can hear you now: "Hey, wait, isn't something missing?" The short answer is not really, because the analog functions make use of very specific hardware in the Arduino microcontroller, there is no need to set them up using the `pinMode()` function inside our `setup()` function. If we try to send an analog signal to a pin that isn't made for it, it simply won't work exactly as expected. So then all we are left with is a function that begins a communication protocol that we can use to let the Arduino talk to us.

```
analogValue = analogRead(sensorPin);
analogValue = constrain(analogValue, minReading, maxReading);
analogValue = map(analogValue, minReading, maxReading, 0, 255);
```

The first three lines of the `loop()` function are where the magic happens in this sketch. The first line obtains a numerical value from the analog input pin correlating to the voltage our sensor is putting out, which should indicate a general presence of a breath. The second line lobs off unusable values at either